

netsim User Manual

Convergence-First Network Simulator

Simon Knight

Adelaide, Australia

March 2026

About this manual. This manual covers running simulations, daemon mode, interactive device inspection, chaos engineering, and convergence testing for 15 protocol families including OSPF v2/v3, IS-IS, BGP (EVPN/L3VPN), MPLS/LDP, RSVP-TE, BFD, LACP, LLDP, VXLAN, RIPv2/RIPng, STP, VRRP v2/v3, and DHCP Relay. Manual ID: NETSIM-UM-001.

Contents

1	Quick Start	1
2	Installation & Prerequisites	2
2.1	Installing the Pre-built Binary	2
2.2	Building from Source with Cargo	2
2.3	Verifying the Installation	2
3	Core Workflows	3
3.1	Running a Simulation	3
3.2	Daemon Mode	3
3.3	Interactive Device Inspection	4
3.4	Convergence Testing	5
3.5	Chaos Engineering	6
3.6	Traffic Analysis	7
4	YAML Topology Reference	8
4.1	Top-Level Structure	8
4.2	Devices	8
4.3	Links and Impairments	9
4.4	Protocol Configuration Blocks	9
4.4.1	OSPF	9
4.4.2	BGP	10
4.4.3	IS-IS	10
4.5	Events and Assertions	10
4.6	Script Section	11
5	CLI Command Reference	11
5.1	Simulation Commands	11
5.2	Daemon Commands	11
5.3	Interactive Commands	12
5.4	Analysis Commands	12
6	Interactive REPL Commands	13
6.1	Show Commands	13
7	Protocol Coverage	14
8	Convergence Detection	15
8.1	The 11 Stability Flags	15
8.2	N-Tick Sustained Stability	16
8.3	Convergence-Relative Scheduling	16
9	Troubleshooting	17
10	Integration	18
10.1	TopoGen — Topology Generation	18
10.2	AutoNetKit — Configuration Generation and Validation	18
10.3	NetCfg — Configuration Compilation	19
10.4	NetVis — Visualising Simulation Results	19
10.5	Workbench — Orchestration	19

11 Further Reading

19

1 Quick Start

Get a two-router OSPF simulation running and asserting reachability in under five minutes.

Step 1. Write a minimal topology file `topology.yaml`:

```
name: two-routers-ospf
tick_ms: 1

devices:
  - id: r1
    type: router
    interfaces:
      - id: eth0
        ip: 10.0.0.1/30
      - id: lo0
        ip: 192.168.1.1/32
  - id: r2
    type: router
    interfaces:
      - id: eth0
        ip: 10.0.0.2/30
      - id: lo0
        ip: 192.168.2.1/32

links:
  - src: r1:eth0
    dst: r2:eth0

ospf:
  area: 0
  devices: [r1, r2]

assertions:
  - type: reachability
    source: r1
    destination: 192.168.2.1
    expected: true
  - type: ospf_neighbor_state
    router: r1
    interface: eth0
    expected_state: Full
```

Step 2. Run the simulation:

```
netsim run topology.yaml
```

`netsim` exits with code 0 if all assertions pass. You should see output similar to:

```
[INFO ] build: 2 routers, 1 link, 2 assertions
[INFO ] converged at tick 47 (47ms)
[PASS ] reachability: r1 -> 192.168.2.1
[PASS ] ospf_neighbor_state: r1/eth0 Full
[INFO ] simulation complete -- 2/2 assertions passed
```

Step 3. Inspect the routing matrix in JSON:

```
netsim run topology.yaml --format json --output results.json
```

Step 4. Explore a richer built-in example:

```
netsim example ospf-triangle
```

This prints a three-router OSPF topology with area 0, loopback reachability assertions, and a link-failure chaos event. Redirect to a file and run it:

```
netsim example ospf-triangle > ospf-triangle.yaml  
netsim run ospf-triangle.yaml
```

Tip

Run `netsim example --list` to see all 40+ built-in examples grouped by protocol family. Each example is a complete, runnable YAML file you can use as a starting point.

2 Installation & Prerequisites

▷ Prerequisites

- A 64-bit Linux or macOS system (daemon mode requires Unix Domain Socket support; Windows is not supported)
- 2 GB RAM minimum; 8 GB recommended for topologies near the 486-device scale ceiling
- No external network access required — netsim is fully self-contained

2.1 Installing the Pre-built Binary

Download the latest release binary from the project release page and place it on your PATH:

```
# Download and install  
curl -Lo netsim https://releases.example.com/netsim/v1.0/netsim-linux-x86_64  
chmod +x netsim  
sudo mv netsim /usr/local/bin/  
  
# Verify  
netsim --version
```

2.2 Building from Source with Cargo

```
cargo install netsim --version 1.0
```

Rust 1.75 or later is required. The build pulls no C library dependencies; the entire simulation engine compiles to a single static binary.

```
git clone https://github.com/example/netsim.git  
cd netsim  
cargo build --release  
./target/release/netsim --version
```

2.3 Verifying the Installation

```
netsim example ospf-triangle | netsim run --stdin
```

A successful install prints `simulation complete` with all assertions passing.

Every release ships with 40+ example topologies covering all 15 protocol families. They are embedded in the binary — no separate download required. Use `netnsim example <name>` to print any of them to stdout.

Scale Ceiling

`netnsim` has been tested and validated to **486 devices**. Topologies larger than ~500 devices are not recommended. Memory consumption grows linearly with device count; a 486-device topology with full BGP EVPN uses approximately 1.2 GB of RAM at peak convergence. For larger networks, validate representative subsets rather than the full topology.

3 Core Workflows

3.1 Running a Simulation

Step 1. Write or obtain a YAML topology file (see section 4 for the full schema reference).

Step 2. Validate the file before running (optional but recommended):

```
netnsim validate topology.yaml
```

Validation checks YAML syntax, schema correctness, IP address formatting, and cross-references (e.g. links referencing non-existent devices) without executing the simulation.

Step 3. Run the simulation:

```
netnsim run topology.yaml
```

Key options:

```
# Output full results as JSON
netnsim run topology.yaml --format json --output results.json

# Cap simulation at 10000 ticks (10 seconds at 1ms/tick)
netnsim run topology.yaml --max-ticks 10000

# Tighten convergence requirement (5 consecutive stable ticks)
netnsim run topology.yaml --convergence-threshold 5

# Capture per-interface PCAP traces
netnsim run topology.yaml --pcap --output pcap/

# Run verbose: print tick-by-tick convergence progress
netnsim run topology.yaml --verbose
```

Step 4. Check the exit code in CI:

```
netnsim run topology.yaml && echo "PASS" || echo "FAIL"
```

`netnsim` exits with code 0 only if all assertions pass and convergence is detected within the tick limit.

Tip

Use `--format json` in CI pipelines. The JSON output includes a per-assertion pass/fail record, the convergence tick, per-protocol stability timestamps, and the full routing matrix. This data is consumable by NetVis and the Workbench orchestrator.

3.2 Daemon Mode

Daemon mode runs a simulation as a persistent background process. You can attach to live devices, inspect protocol state, and inject failures without stopping the simulation.

Step 1. Start a daemon:

```
netnsim daemon start topology.yaml --name my-fabric
```

netnsim run is the primary batch workflow. It exits 0 on success, non-zero on assertion failures.

Daemon sockets live in .netnsim/<name>.sock. All daemons on the same machine share that directory.

Options:

```
# Run at 100ms per tick (slower, less CPU)
netsim daemon start topology.yaml --name lab --tick-interval 100ms

# Start with full protocol debug logging
netsim daemon start topology.yaml --name lab --log-level protocol_debug
```

Step 2. Check daemon status:

```
netsim daemon status my-fabric
```

Output includes PID, uptime, current tick, convergence status, and average tick processing time.

Step 3. List all running daemons:

```
netsim daemon list
```

Use `--clean` to automatically remove stale entries for daemons that have exited.

Step 4. Adjust log verbosity at runtime without restarting:

```
# Switch to protocol-level debug logging
netsim daemon log my-fabric protocol_debug

# Return to normal logging
netsim daemon log my-fabric normal

# Enable full trace (all modules)
netsim daemon log my-fabric full_trace
```

Step 5. Stop the daemon:

```
netsim daemon stop my-fabric
```

Daemon mode requires Unix

The daemon communicates over a Unix Domain Socket (UDS). This requires a Unix-like operating system (Linux or macOS). Windows is not supported for daemon mode.

3.3 Interactive Device Inspection

The `attach` command opens an IOS-style REPL on a live simulated device. Network engineers familiar with Cisco IOS can use the same command vocabulary they already know.

Step 1. Start a daemon (see section 3.2).

Step 2. Attach to a specific device:

```
netsim attach my-fabric r1
```

You are dropped into an IOS-style prompt:

```
r1#
```

Step 3. Inspect the routing table:

```
r1# show ip route
r1# show ip route vrf CUSTOMER-A
r1# show ipv6 route
```

Step 4. Inspect OSPF state:

```
r1# show ospf neighbors
```

```
r1# show ospf database
```

Step 5. Inspect BGP:

```
r1# show bgp summary
r1# show bgp vpnv4 vrf CUSTOMER-A
r1# show bgp evpn
```

Step 6. Test reachability with ping:

```
r1# ping 10.0.1.1
r1# ping 10.0.1.1 -c 5
```

ping drives re-entrant simulation stepping — it injects real ICMP packets into the tick loop and reports actual round-trip behaviour, not just a routing-table lookup.

Step 7. Trace the forwarding path:

```
r1# traceroute 192.168.5.1
r1# show trace 192.168.5.1
```

Step 8. Exit the REPL:

```
r1# exit
```

Tab Completion and Prefix Matching

The REPL supports tab completion and IOS-style prefix matching. Type `sh ip ro` and press Tab — it expands to `show ip route`. Interface names (`eth0`, `eth1`, ...) are completed dynamically from the device's actual interfaces. Type `help` at any time for a full command listing.

For non-interactive scripting, use `exec` to run a single command and capture its output:

```
netsim exec my-fabric r1 "show bgp summary"
netsim exec my-fabric r1 "show ip route" --format json
```

3.4 Convergence Testing

`netsim` treats convergence as a first-class time coordinate rather than a timeout. You can write assertions and schedule events *relative to* the moment convergence is detected.

Step 1. Define a topology with convergence-relative events and assertions in `topology.yaml`:

```
name: ospf-convergence-test
devices: [...]
links: [...]
ospf:
  area: 0
  devices: [r1, r2, r3]

events:
  # Fail a link 500ms after convergence
  - at: converged + 500ms
    action: link_down
    link: r1-r2

  # Check BGP state 100 ticks after convergence
  - at: converged + 100
    device: r1
    command: show bgp summary

assertions:
```

```
# Fail if initial convergence takes more than 200 ticks
- type: convergence_time
  max_ticks: 200

# After link failure reconverges, r1 must still reach r3
- type: reachability
  source: r1
  destination: r3
  expected: true
  at: re-converged
```

Step 2. Run and observe:

```
netsim run topology.yaml --verbose
```

netsim will report the exact tick at which initial convergence was detected, then fire the link-down event, wait for reconvergence, and evaluate assertions.

Step 3. In a CI pipeline, assert on convergence time:

```
netsim run topology.yaml --format json | \
jq '.convergence_tick <= 200'
```

Convergence-Relative Scheduling

Use at: converged + Nms for millisecond offsets or at: converged + N for tick offsets. Both forms work in the events and script sections. This makes tests robust: they do not depend on absolute tick counts, so they pass equally fast on different hardware.

3.5 Chaos Engineering

netsim supports four failure pattern types for automated resilience testing. All patterns are deterministic and reproducible.

Step 1. Define failure patterns in your topology file:

```
failure_patterns:
  # Periodic link oscillation (every 1000 ticks: 200 down, 800 up)
  - name: flap-core-link
    type: periodic
    target: link
    id: r1-r2
    down_ticks: 200
    up_ticks: 800
    start_tick: 100

  # Probabilistic device failure
  - name: random-spine-failure
    type: probabilistic
    target: device
    id: spine1
    check_interval: 500
    fail_probability: 0.05
    recovery_ticks: 300

  # Rapid flapping stress test
  - name: stress-flap
    type: flapping
    target: link
    id: r2-r3
    cycles: 10
    cycle_ticks: 50
    start_tick: 500
```

```
# Cascade: when core link fails, access links also fail
- name: cascade-failure
  type: cascade
  trigger: link-core-down
  dependents: [access-r1-h1, access-r1-h2]
  delay_ticks: 10
```

Step 2. Combine failure patterns with convergence-relative assertions to measure resilience:

```
assertions:
- type: convergence_time
  max_ticks: 150
  label: "reconvergence-after-flap"

- type: reachability
  source: h1
  destination: h2
  expected: true
  at: re-converged
```

Step 3. Run the resilience test:

```
netsim run chaos-topology.yaml --format json
```

Cascade Patterns Require Careful Design

Cascade failure patterns can interact unexpectedly with convergence detection. If a cascade fires before initial convergence is reached, the simulation may oscillate. Use `start_tick` to ensure cascade triggers fire after a known stable state.

3.6 Traffic Analysis

netsim models traffic generators that inject frames at a configured rate. After convergence you can extract the routing matrix and analyse link utilisation.

Step 1. Define traffic generators in the topology:

```
traffic:
- id: tg-web
  src: host-client
  dst: host-server
  rate_mbps: 100
  start_tick: 50

- id: tg-db
  src: host-app
  dst: host-db
  rate_mbps: 50
  start_tick: 50
```

Step 2. Extract the routing matrix from a running daemon:

```
netsim routing-matrix my-fabric --output matrix.json
```

Step 3. Analyse link capacity utilisation:

```
netsim capacity topology.yaml matrix.json
```

The capacity report shows per-interface utilisation as a percentage of configured bandwidth, highlighting over-subscribed links.

Step 4. Compare routing matrices before and after a change:

```
netsim routing-matrix my-fabric --output before.json
# ... make a topology change and restart the daemon ...
netsim routing-matrix my-fabric --output after.json
netsim diff before.json after.json
```

4 YAML Topology Reference

A netsim topology is a single YAML file. This section documents every top-level section and the most important fields.

4.1 Top-Level Structure

```
name: "my-topology" # required
tick_ms: 1          # optional; default 1ms per tick

devices: [...]      # required
links: [...]         # required

ospf: {...}         # optional protocol blocks
bgp: {...}
isis: {...}
mpls: {...}
rsvp: {...}
rip: {...}
stp: {...}
vrrp: {...}

traffic: [...]       # optional
impairment_profiles: [...] # optional
failure_patterns: [...] # optional
events: [...]        # optional
assertions: [...]    # optional
script: [...]        # optional
```

4.2 Devices

```
devices:
- id: r1
  type: router          # router | host | switch | hub
  tier: spine            # optional; used by pattern selectors
  interfaces:
    - id: eth0
      ip: 10.0.0.1/30
      mac: "aa:bb:cc:dd:ee:01" # optional; auto-assigned if omitted
    - id: lo0
      ip: 192.168.1.1/32
  vrfs:                 # optional; defines VRF instances
    - name: CUSTOMER-A
      rd: "65000:100"
      rt_import: ["65000:100"]
      rt_export: ["65000:100"]

- id: h1
  type: host
  interfaces:
    - id: eth0
      ip: 10.1.1.10/24
      gateway: 10.1.1.1

- id: sw1
  type: switch
  vlans: [10, 20, 30]
```

Device types and their capabilities are summarised in table 1.

Table 1: Device types in netsim.

Type	Capabilities
router	Full L3 device. Runs OSPF, BGP, IS-IS, RIP, MPLS, RSVP-TE, BFD, LACP, LLDP, VRRP, DHCP Relay, STP bridge domains, and VRFs
host	L3 endpoint with default gateway. Generates and sinks traffic. Supports ping and traceroute
switch	L2 device with MAC learning, VLAN tagging, and STP
hub	L1 device; floods all frames to all ports

4.3 Links and Impairments

```
links:
- src: r1:eth0
  dst: r2:eth0
  # Optional inline impairments:
  latency_ms: 5
  loss_percent: 0.1
  jitter_ms: 1
  capacity_mbps: 1000
  # Or reference a named impairment profile:
  impairment_profile: wan-latency
```

Named impairment profiles can be shared across many links:

```
impairment_profiles:
- name: wan-latency
  selector:
    by_tier: [edge, core]
  latency_ms: 10
  loss_percent: 0.01
  jitter_ms: 2
- name: access-links
  selector:
    name_pattern: "access-*"
  latency_ms: 1
  capacity_mbps: 100
```

4.4 Protocol Configuration Blocks

4.4.1 OSPF

```
ospf:
  area: 0
  devices: [r1, r2, r3]
  # Area-specific overrides:
  areas:
    - id: 1
      type: stub
      devices: [r4, r5]
  # Per-device cost override:
  interface_costs:
    - device: r1
      interface: eth0
```

```
cost: 10
```

4.4.2 BGP

```
bgp:
  asn: 65000
  route_reflectors: [rr1, rr2]
  peers:
    - local: pe1
      remote: rr1
      type: ibgp
    - local: pe1
      remote: pe2-external
      type: ebgp
      remote_asn: 65001
  # EVPN address family
  evpn:
    enabled: true
    vnis: [10010, 10020]
```

4.4.3 IS-IS

```
isis:
  level: l2 # l1 / l2 / l1l2
  devices: [r1, r2, r3]
  wide_metrics: true
  net_prefix: "49.0001"
```

4.5 Events and Assertions

```
events:
  # Absolute tick
  - at: 1000
    action: link_down
    link: r1-r2

  # Convergence-relative
  - at: converged + 500ms
    action: link_up
    link: r1-r2

  # Command injection
  - at: converged + 100
    device: r1
    command: show bgp summary

assertions:
  - type: reachability
    source: h1
    destination: 192.168.2.1
    expected: true

  - type: convergence_time
    max_ticks: 200

  - type: ospf_neighbor_state
    router: r1
    interface: eth0
    expected_state: Full

  - type: route_present
    router: r1
    prefix: 10.0.0.0/8
    protocol: bgp
```

4.6 Script Section

The script section runs IOS-style commands at convergence-relative times and captures their output for offline analysis:

```
script:
- at: converged
  device: r1
  run: show ip route

- at: converged + 200ms
  device: pe1
  run: show bgp vpnv4 vrf CUSTOMER-A
```

Script output is included in the JSON simulation report.

5 CLI Command Reference

5.1 Simulation Commands

Simulation

Command	Description
<code>netsim run <file></code>	Execute a topology to completion
<code>netsim validate <file></code>	Validate YAML without running
<code>netsim example <name></code>	Print a built-in example to stdout
<code>netsim example --list</code>	List all built-in example names

run Options

Command	Description
<code>--format <ascii json></code>	Output format (default: ascii)
<code>--output <file></code>	Write results to file instead of stdout
<code>--max-ticks <N></code>	Stop after N ticks (default: 100000)
<code>--convergence-threshold <N></code>	Require N consecutive stable ticks (default: 3)
<code>--pcap</code>	Capture per-interface PCAP traces
<code>--verbose</code>	Print tick-by-tick convergence progress
<code>--scenario <name></code>	Select a named scenario variant
<code>--stdin</code>	Read topology YAML from stdin

5.2 Daemon Commands

Daemon

Command	Description
<code>netsim daemon start <file></code>	Start a background daemon
<code>netsim daemon stop <name></code>	Stop a running daemon
<code>netsim daemon status <name></code>	Query live daemon metrics
<code>netsim daemon list</code>	List all running daemons
<code>netsim daemon log <name> <preset></code>	Change log verbosity at runtime

daemon start Options

Command	Description
--name <name>	Daemon name (default: topology basename)
--tick-interval <ms>	Tick duration: 1ms, 50ms, 100ms, 1s
--log-level <preset>	Initial log preset (see table 2)

Log presets accepted by daemon log and --log-level:

Table 2: Log verbosity presets.

Preset	Effect
normal	info level — standard operational output
protocol_debug	OSPF, BGP, IS-IS, BFD at debug; all other modules at info
full_trace	trace for all modules — maximum verbosity; generates large log files

5.3 Interactive Commands

Interactive

Command	Description
netsim attach <daemon> <node>	Open an IOS-style REPL on a device
netsim exec <daemon> <node> "<cmd>"	Run a single command non-interactively
netsim	Launch TUI daemon/device selector

exec Options

Command	Description
--format <ascii json>	Output format for exec results

5.4 Analysis Commands

Analysis

Command	Description
netsim routing-matrix <daemon>	Extract full routing matrix as JSON
netsim diff <a.json> <b.json>	Compare two routing matrices
netsim capacity <topo> <matrix>	Analyse link utilisation
netsim trace <daemon> <src> <dst>	Trace forwarding path hop-by-hop
netsim fib <daemon>	Export FIB in netauto/fib/v1.0 schema

routing-matrix / fib Options

Command	Description
--output <file>	Write to file instead of stdout
--format <json ascii>	Output format

6 Interactive REPL Commands

The interactive REPL supports over 50 IOS-style commands. Partial prefix matching is supported throughout: `sh ip ro` expands to `show ip route`.

6.1 Show Commands

Routing and Interfaces

Command	Description
<code>show ip route</code>	Display IPv4 FIB
<code>show ip route vrf <NAME></code>	Display per-VRF FIB
<code>show ipv6 route</code>	Display IPv6 FIB
<code>show interfaces</code>	Display interface status and counters
<code>show arp</code>	Display ARP table
<code>show vrf [all detail <NAME>]</code>	Display VRF summary or detail

OSPF

Command	Description
<code>show ospf neighbors</code>	OSPF adjacency table
<code>show ospf database</code>	OSPF link-state database
<code>show ospfv3 neighbors</code>	OSPFv3 adjacency table
<code>show ospfv3 database</code>	OSPFv3 LSDB
<code>show ospfv3 route</code>	OSPFv3 computed routes

BGP and MPLS

Command	Description
<code>show bgp summary</code>	BGP neighbour summary
<code>show bgp neighbors</code>	Detailed BGP neighbour state
<code>show bgp vpn</code>	VPNv4/VPNv6 routes
<code>show bgp evpn</code>	EVPN route table
<code>show bgp ipv6 unicast</code>	BGP IPv6 unicast routes
<code>show mpls forwarding</code>	MPLS forwarding table (LFIB)
<code>show mpls ldp bindings</code>	LDP label bindings
<code>show rsvp tunnels</code>	RSVP-TE tunnel state

IS-IS, RIP, and Infrastructure

Command	Description
show isis neighbors	IS-IS adjacency table
show isis database	IS-IS link-state database
show isis route	IS-IS computed routes
show rip database	RIP routing table
show stp bridge	STP bridge state and root information
show stp port	STP per-port role and state
show vrrp brief	VRRP group summary
show vrrp detail	VRRP detailed state including priorities
show lacp details	LACP partner state and LAG membership
show lldp neighbors	LLDP neighbour discovery table
show bfd sessions	BFD session state and timers
show vxlan vtep	VXLAN VTEP endpoint table
show ip dhcp relay	DHCP relay statistics and helper addresses

Diagnostics and Session

Command	Description
ping <target> [-c <COUNT>]	Send ICMP probes (re-entrant tick stepping)
traceroute <target> [-m <TTL>]	Trace hop-by-hop forwarding path
dig <hostname> [@server]	DNS lookup within the simulation
iperf <target> [-d <DUR>]	Measure simulated throughput
show trace <destination>	Static forwarding path analysis
show routing-matrix	Dump full FIB for all destinations
help	List all available commands
exit / quit	Return to shell

7 Protocol Coverage

netsim implements 15 protocol families. All implementations target RFC-correct behaviour at tick granularity, not vendor-specific quirks. All protocol timers are converted to integer tick counts at build time.

Table 3: Protocol families supported by netsim v1.0.

Protocol	Version(s)	Key Features
OSPF	v2 (RFC 2328), v3 (RFC 5340)	Areas, stub/NSSA, virtual links, SPF with incremental recalculation; OSPFv3 supports IPv6 with link-local addressing
IS-IS	ISO 10589	L1/L2/L1L2, wide metrics, TLV extensibility
BGP-4	RFC 4271	iBGP, eBGP, route reflection, confederations, communities, AS-path filtering
BGP EVPN	RFC 7432	Type-2/3/5 routes, RT import/export, ARP suppression, multi-homing
MPLS/LDP	RFC 5036	Label distribution, penultimate hop popping, ECMP label stacking
RSVP-TE	RFC 3209	Explicit paths, bandwidth reservation, fast reroute
BFD	RFC 5880	Asynchronous mode; microsecond-granularity timers mapped to ticks
LACP	802.3ad	Fast (100 ms) and Slow (3 s) timers, LAG bundling, actor/partner state machines
LLDP	802.1AB	Periodic hellos, neighbour table, TLV parsing
VXLAN	RFC 7348	Encap/decap, head-end replication
RIPv2 / RIPv6	RFC 2453 / RFC 2080	Periodic and triggered updates, split horizon, 15-hop limit, route poisoning; RIPv6 adds IPv6
STP	802.1D	Root bridge election, port roles (Root/Designated/Blocking), topology change detection
VRRP	v2 RFC 3768, v3 RFC 5798	Master/Backup election, virtual IP, tracked interfaces, priority decrements, virtual MAC 00:00:5E:00:01:VRID
DHCP Relay	RFC 2131	Broadcast-to-unicast, multiple helper addresses, GIADDR population, Option 82

Tip

RIP, STP, VRRP, and DHCP Relay do not require separate convergence flags. RIP installs into the RIB (tracked by FIB stability flag #1), STP port state changes affect frame delivery, VRRP failover manifests as a FIB change, and DHCP Relay forwards without modifying routing state. The existing 11-flag convergence predicate covers all four protocols transitively.

8 Convergence Detection

Convergence detection is netsim's most distinctive feature. Rather than stopping after a fixed timeout or waiting for packet quiescence (which never arrives with periodic protocols like BFD and LLDP), netsim defines convergence as the simultaneous stability of 11 independent protocol states.

8.1 The 11 Stability Flags

Convergence is declared when all 11 flags are stable for N consecutive ticks (default $N = 3$):

Table 4: The 11 convergence stability flags.

#	Flag	Stable when...
1	FIB	No FIB entry changed this tick
2	VRF FIBs	No per-VRF FIB entry changed this tick
3	MPLS LFIB	No MPLS label forwarding entry changed
4	LDP bindings	No LDP label binding changed this tick
5	VPNv4 Loc-RIB	No VPNv4 route changed this tick
6	BGP Loc-RIB	No BGP route changed this tick
7	EVPN Loc-RIB	No EVPN route changed this tick
8	OSPF neighbours	No OSPF adjacency changed state
9	BGP sessions	No BGP session changed state this tick
10	RSVP tunnels	No RSVP-TE tunnel changed state
11	IS-IS adjacencies	No IS-IS adjacency changed state

PCAP captures, BMP feeds, and NetFlow are deliberately excluded: they generate continuous background traffic that would prevent convergence from ever being detected. Similarly, LLDP periodic hellos are excluded via `EtherType::is_control_plane() == false`.

8.2 N-Tick Sustained Stability

The N -tick requirement prevents false positives from “transient stability” — a topology that appears converged for one tick between two routing update bursts. The default $N = 3$ is sufficient for all 15 protocol families at tested scale. You can tighten this requirement for high-confidence production validation:

```
netsim run topology.yaml --convergence-threshold 10
```

8.3 Convergence-Relative Scheduling

Once convergence is detected at tick t_c , it becomes a named time anchor. All subsequent events and script entries can reference it:

```
events:
- at: converged + 100    # 100 ticks after convergence
  action: link_down
  link: r1-r2

- at: converged + 500ms  # 500ms (= 500 ticks at 1ms/tick)
  device: PE1
  command: show bgp summary
```

This makes three workflows possible that are impossible with timeout-based simulators:

1. **CI-gatable assertions:** Fail a pipeline if convergence takes longer than N ticks after a link failure.
2. **Convergence-relative chaos:** Inject a second failure 100 ticks after convergence from the first, without hard-coding absolute tick numbers.
3. **Post-convergence inspection:** Run `show bgp summary` on PE1 exactly 500 ms after convergence to verify a stable BGP table.

9 Troubleshooting

Problem 1: Simulation never converges; `-max-ticks` is reached

Cause: A protocol is oscillating (e.g. a BGP route is continuously withdrawn and re-advertised, or a BFD session is flapping due to misconfigured timers). The convergence predicate cannot be satisfied while any of the 11 flags keeps resetting.

Solution: Run with `-verbose` to see which flag is resetting and on which tick. Start the daemon instead (`netsim daemon start`) and attach to the oscillating device (`netsim attach`) to inspect protocol state with `show bgp summary` or `show bfd sessions`. Enable `protocol_debug` logging with `netsim daemon log <name> protocol_debug` for detailed protocol-level traces.

Problem 2: Assertions pass in isolation but fail in a chain-failure scenario

Cause: Convergence-relative assertions use the *latest* convergence event. If a second failure fires before the simulation re-converges, the `at: re-converged` anchor may refer to convergence after the second failure, not the first.

Solution: Use explicit `at: converged + N` offsets for intermediate assertions and reserve `at: re-converged` only for the final steady state. Alternatively, chain failures with sufficient `up_ticks` between oscillations.

Problem 3: `ping` and `traceroute` in the REPL behave differently from a routing-table lookup

Cause: `ping` drives re-entrant tick stepping. It injects real ICMP packets and advances the simulation until replies arrive or a timeout is reached. The route may exist in the FIB but a mis-configured next-hop MAC or a wire with 100% loss will silently drop ICMP packets, causing `ping` to report unreachable.

Solution: Use `show ip route <prefix>` to confirm the FIB entry, then `show trace <destination>` for a static forwarding path analysis that bypasses packet injection. Check link impairments with `show interfaces`.

Problem 4: Vendor-specific behaviour is not reproduced (e.g. Cisco OSPF graceful restart, Juniper route preference differences)

Cause: `netsim` models RFC behaviour, not vendor-specific implementation quirks, defaults, or proprietary extensions. This is a deliberate design decision to preserve determinism.

Solution: Use `netsim` for “does the design work according to the RFCs?” and a container-based emulator such as Containerlab for “does this specific NOS version behave as expected?”. These tools are complementary, not competitive.

Problem 5: JSON output is inconsistent between `netsim run --format json` and `netsim routing-matrix`

Cause: `netsim run --format json` produces the full simulation report (assertions, convergence events, per-protocol stability timeline). `netsim routing-matrix` produces only the current FIB snapshot in the `netauto/fib/v1.0` schema. The two formats have different schemas and are not interchangeable.

Solution: Use `netsim run --format json` for CI assertion checking and pass/fail reporting. Use `netsim routing-matrix` and `netsim fib` for downstream tool integration

(NetVis, Workbench). Refer to the JSON schema documentation in NETSIM-TR-001 Appendix B.

Problem 6: BFD sessions configured with `bfd: true` in YAML are not automatically created between all OSPF neighbours

Cause: The `bfd: true` flag in YAML is not yet fully auto-wired through the Builder. BFD sessions must currently be configured explicitly in the topology. This is a known limitation.

Solution: Declare BFD sessions explicitly: `bfd.sessions: [{local: r1, remote: r2, interval_ms: 300}]`. Verify session state with `show bfd sessions` after convergence. Full auto-wiring from the YAML flag is deferred to a future milestone.

Problem 7: Simulation memory usage grows unexpectedly for large topologies

Cause: PCAP capture (`--pcap`) writes an in-memory ring buffer per interface. At 486-device scale with 630+ interfaces, this can consume several hundred megabytes. BGP EVPN Loc-RIB is also memory-intensive when many VNIs are active.

Solution: Disable PCAP unless needed. For EVPN topologies near the scale ceiling, reduce the number of active VNIs or validate representative subsets. Monitor memory with `netsim daemon status <name>` which reports peak RSS.

10 Integration

netsim is one component in a wider network automation ecosystem. The following tools integrate directly with netsim.

10.1 TopoGen – Topology Generation

TopoGen generates seed-deterministic synthetic topologies. Pipe its output directly into netsim for large-scale automated testing:

```
# Generate a 128-device fat-tree and simulate it
topogen generate --nodes 128 --topology fat-tree --seed 42 \
| netsim run --stdin

# Generate with explicit OSPF and BGP configuration
topogen generate --nodes 64 --protocols ospf,bgp --seed 1 \
> generated.yaml
netsim validate generated.yaml
netsim run generated.yaml --format json
```

See also: *TopoGen User Manual* (TOPOGEN-UM-001) — Seed parameters, topology templates, and protocol configuration presets.

10.2 AutoNetKit – Configuration Generation and Validation

AutoNetKit (ANK) builds type-safe network topology models and generates device configurations. Use netsim to validate that ANK-generated configurations produce the correct routing behaviour before deployment:

```
# Build topology model with ANK
ank build topology.yaml --output ank-output/

# Simulate the same topology with netsim to verify routing intent
netsim run topology.yaml --format json --output sim-results.json
```

See also: *AutoNetKit User Manual* (ANK-UM-001) — Topology modelling, type checking, and export formats.

10.3 NetCfg — Configuration Compilation

NetCfg compiles abstract device models to vendor-specific configurations. Export the netsim FIB as a validation baseline for NetCfg-compiled configurations:

```
# Run simulation and export FIB
netsim run topology.yaml
netsim daemon start topology.yaml --name validation
netsim fib validation --output baseline-fib.json

# Compile configs with NetCfg and validate against baseline
netcfg compile topology.yaml --platform ios
netcfg validate --baseline baseline-fib.json
```

See also: *NetCfg User Manual* (NETCFG-UM-001) — Compilation targets, platform support, and FIB validation integration.

10.4 NetVis — Visualising Simulation Results

NetVis renders netsim routing matrices and convergence timelines as interactive diagrams. Feed it the JSON output from `netsim run`:

```
netsim run topology.yaml --format json --output results.json
netvis render results.json --open
```

NetVis can also connect to a live daemon and display real-time protocol state changes:

```
netvis attach my-fabric --live
```

See also: *NetVis User Manual* (NETVIS-UM-001) — Rendering options, live daemon connection, and export formats.

10.5 Workbench — Orchestration

Workbench orchestrates multi-step workflows that combine topology generation, simulation, configuration compilation, and deployment. A typical CI/CD pipeline might look like:

```
# workbench.yaml defines: topogen -> netsim -> netcfg -> deploy
workbench run pipeline.yaml --env staging
```

netsim participates as a validation stage: Workbench starts the simulation, waits for convergence, evaluates assertions, and only proceeds to the next stage if all assertions pass.

See also: *Workbench User Manual* (WORKBENCH-UM-001) — Pipeline syntax, stage configuration, and rollback policies.

11 Further Reading

- *netsim Technical Report* (NETSIM-TR-001) — Full architecture documentation covering the tick pipeline, convergence predicate design, forwarding architecture (RIB/FIB/LFIB), all 15 protocol implementations in detail, daemon gRPC interface, performance benchmarks at 486-device scale, and known limitations. Recommended reading for engineers extending netsim or integrating it into custom toolchains.
- *YAML Schema Quick Reference* (NETSIM-TR-001 Appendix A) — Complete field-by-field reference for every YAML key, including types, defaults, and constraints.
- *CLI Command Quick Reference* (NETSIM-TR-001 Appendix B) — Full listing of all 50+ interactive REPL commands with syntax and examples.